

State Transition Testing Guide

Test how a system behaves as it moves between states over time.

Introduction

State Transition Testing is a black-box technique for systems whose behaviour depends on their current state and the history of events — login flows, order lifecycles, media players, ATMs. It models the system as states, events and transitions, then designs tests to cover them. This guide gives you the model, a worked example and the coverage levels.

Core Concepts

Term	Meaning
State	A condition the system can be in (e.g. Logged Out, Locked)
Event	An input or trigger (e.g. enter password)
Transition	The move from one state to another caused by an event
Action	What the system does during a transition (optional)
Guard	A condition that must be true for a transition to fire (optional)

Two Ways to Model It

- State diagram — a graph of states (boxes) and transitions (arrows) labelled with the triggering event; best for visualising behaviour.
- State table — a grid of states versus events showing the resulting state; best for spotting invalid or missing transitions.

Worked Example — login with account lock (state diagram described)

States: Logged Out, Logged In, Locked. From Logged Out, a correct password transitions to Logged In. An incorrect password stays in Logged Out but increments a counter; after 3 failed attempts the guard fires and the system moves to Locked. From Logged In, a logout event returns to Logged Out. From Locked, only a reset event returns to Logged Out. Picture three boxes with arrows: Logged Out to Logged In (correct password), Logged Out to itself (wrong password, attempts < 3), Logged Out to Locked (wrong password, 3rd attempt), Logged In to Logged Out (logout), Locked to Logged Out (reset).

The Same System as a State Table

Current State	Event: correct pwd	Event: wrong pwd	Event: logout	Event: reset
Logged Out	Logged In	Logged Out / Locked	- (invalid)	- (invalid)
Logged In	- (invalid)	- (invalid)	Logged Out	- (invalid)
Locked	- (invalid)	- (invalid)	- (invalid)	Logged Out

The blank/"invalid" cells are gold: they are transitions the specification may not have defined. Testing them (what happens if you send "logout" while Locked?) often finds real defects.

Coverage Levels

Coverage	What it requires
All states (0-switch, state coverage)	Visit every state at least once
All valid transitions (0-switch, transition coverage)	Exercise every valid transition at least once — the usual ISTQB minimum
All transitions incl. invalid	Also attempt every invalid event in every state
N-switch (e.g. 1-switch)	Cover every sequence of N+1 successive transitions

Exam Traps & Common Mistakes

- Confusing state coverage (visit every state) with transition coverage (exercise every transition) — transition coverage is stronger.
- A 1-switch test covers pairs of transitions, not single ones — do not equate "1-switch" with "one transition".
- Ignoring invalid transitions, which is exactly where robustness defects hide.
- Forgetting guards — a transition may only fire when a condition (like attempts = 3) is met.

WATCH OUT

Achieving "all states" coverage does NOT guarantee "all transitions" coverage. You can touch every state while missing several transitions between them. Transition coverage is the more meaningful target.

Best Practices

- Draw the diagram first, then derive the table to catch missing transitions.
- Always include invalid-event tests for robustness.
- Aim for at least all-valid-transition coverage as a baseline.

INSIDE STLC PRO TIP

The invalid cells in a state table are the most valuable tests you will write. Specifications rarely define what happens when an unexpected event arrives in a given state — so that is where the defects live.